SLAM Robot

Abstract

The project consists of a simulated TurtleBot3-Waffle-style robot maps a custom Gazebo world with SLAM, localises on that map, drives wherever you click in RViz, and recognises red and blue objects with a neural network which was made by fine tuning a pre-trained YOLO26 Model. Everything is available in one Docker image making it easy for anyone to use.

Built Details

- A custom Gazebo World.
- A Robot model derived from the TurtleBot3 Waffle. The robot carries a 5 Hz LiDAR, a 640 x 480 RGB-D camera.
- A map made by SLAM Toolbox.
- Autonomous navigation with Nav2: AMCL localization, costmaps, a global planner, the DWB local planner and a behaviour tree.
- A fine-tuned YOLO26n detector.
- A Docker image.

Simulated World & Robot

A Gazebo simulator was used to design the World. First a simple outline of the world was created, then it was filled with models then the world was further improved through changing the code.

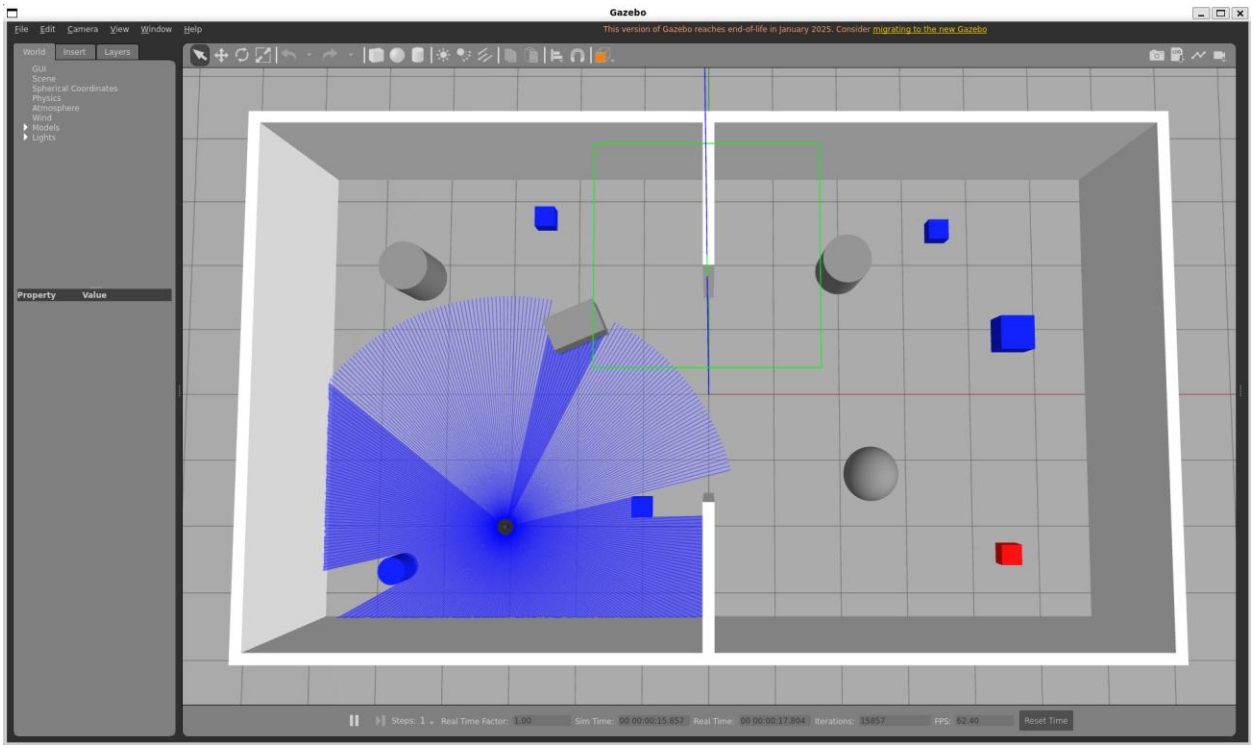


Fig: The Gazebo World

The real-time factor (RTF) says how fast simulated time passes compared with the wall clock. The world uses ODE with a fixed time step h = 0.001s and asks for 1000 physics steps per second, f is the number of steps actually computed per wall-clock second.

$$RTF = \frac{\Delta t_{sim}}{\Delta t_{wall}} = h \cdot f$$

On an average I got an RTF of around 0.85-0.90.

ODE Advancement

At every step ODE updates velocities first and positions afterwards (a *semi-implicit Euler* scheme). Collect the velocities of all bodies in a vector **v**. Let **M** be the mass matrix, **f** the external forces, and let joints and contacts be written as constraints with Jacobian **J** and unknown constraint forces **λ**:

$$\boldsymbol{M(v_{k+1} - v_k) = hf_k + J^T\lambda, \qquad x_{k+1} = x_k \oplus hv_{k+1}}$$

The change of velocity is the push of the outside forces plus the push of the constraints (a wheel cannot sink into the floor; a joint cannot come apart). The positions then move with the new velocity. The constraint forces λ are chosen so that the constrained velocities satisfy $Jv_{k+1} = c$, where c includes a correction of the position error.

Sensors

The Robot uses 3 sensors:

- LIDAR: LiDAR measures distance by sending out a narrow beam and timing the return.

  Lidar Reading:

  $$\boldsymbol{r_i = min\{t > 0 : p + td_i \; lies \; in \; the \; surface\} + \varepsilon_i}$$

  p is the sensor position

  t is the distance along the ray

  $$\boldsymbol{d_i = \begin{pmatrix} cos\varphi_i \\ sin\varphi_i \end{pmatrix}}$$

  d_i is the direction of the beam & φ is the beam angle

  ε_i is the measurement noise
- Camera: A 640 x 480 RGB-D Camera. Its horizontal field of view α_h and the image width W fix the focal length in pixels. The picture's aspect ratio then fixes the vertical field of view.

  Focal Length:

  $$\boldsymbol{f = \frac{W}{2tan(\frac{\alpha_h}{2})}}$$

  Vertical Field of View:

  $$\boldsymbol{\alpha_v = 2arctan\left(tan\frac{\alpha_h}{2} \cdot \frac{H}{W}\right)}$$
- IMU: The IMU publishes angular velocity and linear acceleration. The IMU was later not used for navigation.

Robot Parameters

Parameters	**Values**
Wheel Base	0.287m
Wheel Diameter	0.066m
LiDAR	360 beams, 5 Hz, 0.12 to 3.5 m, σ = 1 cm
Camera	f = 565.6 px, field of view 59.0°. Rate: (Nominal: 15Hz, Measured: 11-12 Hz)

Coordinate Frames, Motion & Odometry

Coordinate Frames and Transforms

A robot at position (x, y) with heading θ defines a frame with its own axes. A point that has coordinates (p_x, p_y) in the robot frame has these coordinates in the world frame:

$$\begin{pmatrix} X \\ Y \end{pmatrix} = \begin{pmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{pmatrix} \begin{pmatrix} p_x \\ p_y \end{pmatrix} + \begin{pmatrix} x \\ y \end{pmatrix}$$

If the robot makes a relative move $(\Delta x, \Delta y, \Delta\theta)$, its new position in the world is:

$$\begin{pmatrix} x' \\ y' \\ \theta' \end{pmatrix} = \begin{pmatrix} x + \Delta x cos\theta - \Delta y sin\theta \\ y + \Delta x sin\theta + \Delta y cos\theta \\ \theta + \Delta\theta \end{pmatrix}$$

Robot Movement with respect to wheel

The robot has Two drive wheels in an axle a distance b apart. Each wheel can be driven independently.

- v_R, v_L are the wheel speeds related to the ground.
- $\dot{\varphi}_r, \dot{\varphi}_l$ are the angular velocity.

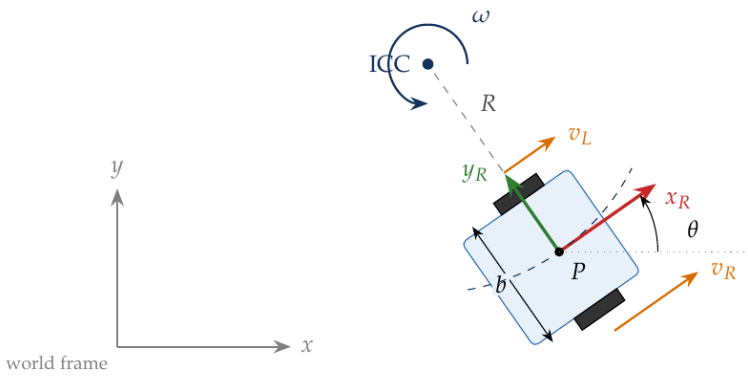


Fig: Robot's Frame and the World Frame

When the wheels move at same speed the robot moves forward or backwards. When they move at different speeds the robot turns. Because the wheels cannot slip sideways, the robot turns as a rigid body about one point on the axle line, called the Instantaneous Center of Curvature (ICC). Let R be the signed distance from the robot's centre P to the ICC (positive to the left), and let ω be the turning rate.

A point at distance ρ from the turning center moves at speedω_ρ. The robot centre is at distance R from the ICC, the right wheel at R + b/2 and the left wheel at R - b/2.

$$\boldsymbol{v = \omega R v_R = \omega\left(R + \frac{b}{2}\right) v_L = \omega\left(R - \frac{b}{2}\right)}$$

Solving them we get,

$$\boldsymbol{v = \frac{v_R + v_L}{2}, \omega = \frac{v_R - v_L}{b} \Leftrightarrow v_R = v + \frac{\omega b}{2}, v_L = v - \frac{\omega b}{2}}$$

Odometry

Odometry estimates where the robot is by adding up how far each wheel has rolled. It starts from a known pose and updates it a little at a time, 30 times per second in this robot. Over one step each wheel rolls a distance $\boldsymbol{\Delta s_R = r\Delta\varphi_R}$ and $\boldsymbol{\Delta s_L = r\Delta\varphi_L}$. Using the previous equations we get,

$$\boldsymbol{\Delta s = \frac{\Delta s_R + \Delta s_L}{2} \Delta\theta = \frac{\Delta s_R - \Delta s_L}{b}}$$

So, the odometry in the next step will be,

$$\boldsymbol{\begin{pmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{pmatrix} = \begin{pmatrix} x_k + \Delta s \cos\left(\theta_k + \frac{\Delta\theta}{2}\right) \\ y_k + \Delta s \sin\left(\theta_k + \frac{\Delta\theta}{2}\right) \\ \theta_k + \Delta\theta \end{pmatrix}}$$

Wheel Noise

Real wheels slip, floors are uneven and encoders have limited resolution, so the measured wheel distances $\Delta s_R, \Delta s_L$ are noisy.

$$\boldsymbol{Q = diag(k|\Delta s_R|, k|\Delta s_L|)}$$

A larger k means worse odometry.

The new uncertainty is the old one propagated through the pose Jacobian, plus the wheel noise Q propagated through the wheel Jacobian. The Covariance Matrix is.

$$\boldsymbol{P_{k+1} = F_\rho P_k F_\rho^T + F_\Delta Q F_\Delta^T}$$

Where the Jacobians (with $\theta' = \theta + \Delta\theta/2$)

$$F_\rho = \begin{bmatrix} 1 & 0 & -\Delta s sin\theta' \\ 0 & 1 & \Delta s cos\theta' \\ 0 & 0 & 1 \end{bmatrix}, \qquad F_\Delta = \begin{bmatrix} \frac{1}{2}\cos\theta' - \frac{\Delta s}{2b}\sin\theta' & \frac{1}{2}\cos\theta' + \frac{\Delta s}{2b}\sin\theta' \\ \frac{1}{2}\sin\theta' + \frac{\Delta s}{2b}\cos\theta' & \frac{1}{2}\sin\theta' - \frac{\Delta s}{2b}\cos\theta' \\ \frac{1}{b} & -\frac{1}{b} \end{bmatrix}$$

What Simulator Publishes

The equations above describe real wheel odometry. The simulator, however, can publish odometry from two sources, selected by the plugin option odometry_source.

Value	Name	How /odom is computed
0	ENCODER	Find using the Odometry Update
1	WORLD	Read from the Physics Engine

Value 1 WORLD was used for this Simulation.

World Odometry Velocity

$$v_x = cos\,\varphi\,\dot{x}_\omega + sin\,\varphi\,\dot{y}_\omega, \qquad v_y = cos\,\varphi\,\dot{y}_\omega - sin\,\varphi\dot{x}_\omega$$

Using World Model gives 4 advantages:

- Mapping becomes easier than a real robot.
- Localization error can be measured. Since the true pose is available as /odom, the difference between AMCL's estimate and /odom is the localisation error of AMCL itself.
- There is no wheel slipping
- The odometry equation was never needed in the code.

The cost: The simulation is easier. Reality is Harder.

Mapping & SLAM

To build a map, the robot must know where it was when it took each measurement. To know where it is, the robot needs a map to compare against. Each needs the other. This is where SLAM (Simultaneous Localization and Mapping) comes in.

SLAM estimates

$$\boldsymbol{p}(\boldsymbol{x}_{1:t}, \boldsymbol{m} | \boldsymbol{z}_{1:t}, \boldsymbol{u}_{1:t})$$

where,

- $x_{1:t} = (x_1, \ldots, x_t)$ is the whole trajectory where $x_k = (x_k, y_k, \theta_k)$ is one pose.
- m is the map.
- $z_{1:t}$ are the measurements from the LIDAR.
- $u_{1:t}$ are the odometry measurements.

Two Solutions:

- Filter Based: Filter-based methods keep only the latest pose and a map, and update them with each new scan.
- Graph Based: Graph-based methods keep every pose as a node in a graph, record how the poses relate to each other as edges, and then solve one large least-squares problem that finds the poses that best satisfy all the edges at once

SLAM Toolbox is graph-based.

SLAM Toolbox Pipeline

- A scan is only made when the robot has moved 0.2 m or turned 0.2 rad since the last scan and at least 0.5s have passed. This makes sure the graph is not filled with identical scans.
- The odometry provides a first guess of where the scan was taken. The matcher searches the neighbourhood of that guess, to find the best fits the recent scans.
- The scan becomes a node. Edges connect it to the scans it was matched against
- If the scan is close in space to an old scan that is far back in the chain, the matcher tries to link them.
- A least-squares solver adjusts all poses to satisfy all edges as well as possible.
- Every beam of every scan is traced from its pose into the grid, and each cell is classified.
- The map goes out on /map every 2 s, and the correction map → odom is broadcast 50 times per second.

Occupancy Grid Maps

An occupancy grid divides the floor into small square cells (in this case 5 cm × 5 cm). Each cell holds one of three states: occupied (an obstacle is there), free (the robot can drive there) or unknown (never observed).

Log-odds and the Probability it encodes

$$\boldsymbol{l(m_i) = ln\frac{p(m_i)}{1-p(m_i)}}, \qquad \boldsymbol{p(m_i) = 1 - \frac{1}{1+e^{l(m_i)}}}$$

Adding evidence from a new measurement z_t

$$\boldsymbol{l_t(m_i) = l_{t-1}(m_i) + ln\frac{p(m_i|z_t)}{1-p(m_i|z_t)} - l_0}$$

Where:

- m_i is cell i
- $p(m_i)$ the probability it is occupied
- l_0 is the log-odds
- $p(m_i|z_t)$ that a single beam assigns the cell

Scan Matching

Scan matching is done by overlaying the new scan on the recent ones and slide and turn it until the points line up. The pose at which they line up best is the pose of the new scan.

- Reference Grid: The points of the earlier scans are marked in a fine grid. Each point is smeared: the cells around it get a value that falls off like a Gaussian.

  $$\boldsymbol{G(q) = 100\ max\ exp\left(-\frac{1}{2}\frac{\|q-h_k\|^2}{\sigma_s^2}\right)}, \qquad \boldsymbol{\sigma_s = 0.1m}$$

  Where q is the cell center, h_k are the hit points of the reference scans, σ_s s the correlation search space smear deviation. A cell holding a hit scores 100, a cell 10 cm from the nearest hit scores $100e^{-0.5} = 61$.
- A candidate pose places the new scan's endpoints in the world. Beam i has range r_i and direction φ_i in the sensor frame:

  $$\boldsymbol{p_i(T) = \begin{pmatrix} x \\ y \end{pmatrix} + \begin{bmatrix} cos\,\theta & -sin\,\theta \\ sin\,\theta & cos\,\theta \end{bmatrix} \begin{pmatrix} r_i\,cos\,\varphi_i \\ r_i\,sin\,\varphi_i \end{pmatrix}}$$
- The response of a candidate pose is

  $$\boldsymbol{\rho(T) = \frac{1}{100n}\sum_{i=1}^{n} G(p_i(T))}$$

It looks at the smeared grid value under every endpoint, average over the n beams, and divide by 100 so that the result lies between 0 and 1.

- Penalized Response:

$$\rho_{pen} = \rho \cdot max\left(1 - 0.2\frac{\Delta x^2 + \Delta y^2}{0.5}, 0.5\right) \cdot max\left(1 - 0.2\frac{\Delta\theta^2}{1}, 0.9\right)$$

Where $(\Delta x, \Delta y, \Delta\theta)$ is the offset of the candidate from the guess. The numbers 0.5 and 1.0 in the denominators are distance_variance_penalty and angle_variance_penalty, the 0.2 are fixed constants in the code, and 0.5 and 0.9 are minimum_distance_penalty and minimum_angle_penalty.

- Matched Pose:

$$\hat{T} = arg\ max\ \rho_{pen}(T)$$

How sure is the pose?

The back end needs to know how much to trust each match. SLAM Toolbox estimates this from the shape of the response surface.

$$\sum_{xx} = \frac{\sum_k \omega_k (x_k - \hat{x})^2}{\sum_k \omega_k}, \sum_{yy} = \frac{\sum_k \omega_k (y_k - \hat{y})^2}{\sum_k \omega_k}, \sum_{xy} = \frac{\sum_k \omega_k (x_k - \hat{x})(y_k - \hat{y})}{\sum_k \omega_k}$$

If the response has a sharp peak, only a few candidates score near the best and the covariance is small meaning the model is sure. If the response is a long ridge (for example when the robot looks along a featureless corridor), many candidates along the ridge score almost as well, and the covariance is large in that direction meaning it can't tell how far it is. This is also why when mapping we should move slowly and turn slowly to make sure we get a good scan.

Pose Graph Optimization

The graph has

- A node $x_i = (x_i, y_i, \theta_i)$ is the pose scan i.
- An edge between nodes i & j stores the measured relative pose $\hat{z}_{ij} = (\hat{x}_{ij}, \hat{y}_{ij}, \hat{\theta}_{ij})$ from scan matching. The Information Matrix derived from the information matrix $\Omega_{ij} = \sum_{ij}^{-1}$ which we get from the covariance.

Error of an edge

$$e_{ij} = \begin{bmatrix} R(\theta_i)^T(p_j - p_i) - \hat{p}_{ij} \\ wrap(\theta_j - \theta_i - \hat{\theta}_{ij}) \end{bmatrix}$$

SLAM Toolbox's solver code contains exactly this expression (the class PoseGraph2dErrorTerm), multiplied by the square root of the information matrix.

Total Cost

$$F(x) = \sum_{(i,j)} e_{ij}{}^{T} \, \Omega_{ij} e_{ij}$$

Each edge contributes its squared error, weighted by how confident it is. The optimiser looks for the poses that make the total as small as possible.

Loop Closure

A loop closure is an edge between two scans that are close in space but far apart in the chain of poses, such as when the robot comes back to a place after a long drive. It is the only thing that can remove accumulated drift. SLAM does multiple tests for this: loop_match_minimum_chain_size, loop_match_minimum_response_coarse.

Parameters and Map Making

The configuration is the stock SLAM Toolbox file, slam_params.yaml with 4 changes: map_update_interval set to 2.0 s, minimum_travel_distance to 0.2 m and minimum_travel_heading to 0.2 rad. This was done so that the map grows quickly.

The simulation was started, then SLAM Toolbox (slam mapping) and the keyboard teleoperation node. The robot was driven through both rooms and around the objects until the map looked complete in RViz. Then the map was saved.

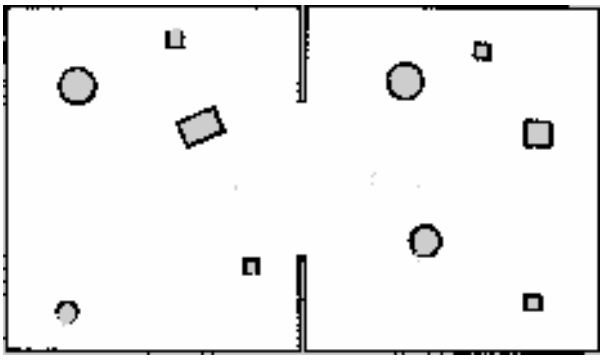


Fig: The Map created by moving the robot throughout the Gazebo World manually

Localization & Navigation

The previous section shows a map is built. Now this section shows how the robot uses the produced map – localization, how it plans a path to a goal, and how it drives that path without hitting any of the obstacles.

The robot has a map. AMCL (adaptive Monte Carlo localisation) keeps a cloud of 500 - 2000 guesses about the robot's pose (green particles). After each move, it shifts every particle the way the odometry says the robot moved, with some random error. After each scan, it asks of every particle: "if the robot were really here, would this scan fit the map?" Particles that fit well are kept and particles that fit badly die out.

Bayes Filter

The robot's knowledge about its pose $\boldsymbol{x_t}$ is a probability distribution, the belief $\boldsymbol{bel(x_t)}$. It is updated in two steps with every control $\boldsymbol{u_t}$ (odometry) and measurement $\boldsymbol{z_t}$ (scan):

$$\boldsymbol{bel(x_t) = \eta \cdot p(z_t|x_t, m) \int p(x_t|x_{t-1}, u_t) bel(x_{t-1})\, dx_{t-1}}$$

$\boldsymbol{m}$ is the map, $\boldsymbol{\eta}$ is a constant that makes the result add up to 1, the integral is the prediction (move the old belief with the motion model) and the product with $\boldsymbol{p(z_t|x_t, m)}$ is the correction.

The integral cannot be solved in closed form for a LiDAR in a real map, so AMCL approximates the belief by particles.

$$\boldsymbol{bel(x_t) \approx \left\{\left(x_t^{[m]}, w_t^{[m]}\right)\right\}_{m=1}^{M}, \qquad x_t^{[m]} = (x, y, \theta), \qquad \sum_m w_t^{[m]} = 1}$$

Sensor Model

AMCL uses a likelihood field model. It takes each beam of the real scan, places its end point in the map as if the robot stood at the particle's pose, and looks up how far that end point is from the nearest wall or object.

$$\boldsymbol{p_i = z_{hit} \cdot e^{\left(-\frac{{d_i}^2}{2{\sigma_{hit}}^2}\right)} + \frac{z_{rand}}{r_{max}}}$$

- $\boldsymbol{d_i}$ is the distance (m) from the end point of beam i to the nearest occupied cell of the map
- $\boldsymbol{\sigma_{hit} = 0.2m}$ is the expected error of a good measurement.
- $\boldsymbol{z_{hit} = 0.5}$ **&** $\boldsymbol{z_{rand} = 0.5}$ are the weights for a good measurement.
- $\boldsymbol{r_{max} = 3.5m}$ is the LiDAR Range.

A beam whose end point lies on a wall scores higher. The farther it is from any wall, the lower the score falls, down to a noise.

The beams are then combined into one weight for the particle.

$$w_t^{[m]} \leftarrow w_t^{[m]} \cdot \left(1 + \sum_i {p_i}^3\right)$$

Cubing makes good beams count more, so a few beams that fit very well outweigh many that fit poorly.

AMCL groups the particles into clusters and publishes the mean pose of the heaviest cluster on /amcl_pose. It turns the estimate into the transform map → odom, so that every other node can find the robot on the map through the transform tree. The transform is post-dated by transform_tolerance = 1.0 s so that it is always valid slightly into the future.

This means that

- No global localization is needed.
- The filter updates after 0.25 m of travel or 0.2 rad of turning, which is about once per second at top speed.
- Localization error can be measured.

Costmap

A costmap is a grid like the map, but each cell holds a cost from 0 (free) to 254 (certain collision). The planner and the controller use costs, not the raw map, because a robot has a size and should not drive grazing a wall even where it would technically fit.

A costmap is built from layers, each adding its own information:

- Static Layer which copies the saved map.
- Obstacle Layer (Voxel Layer for 3D Maps) marks cells where the LiDAR currently sees something and clears cells where it sees nothing.
- Inflation Layer spreads cost outward from every obstacle.

Inflation Function (Nav2 computeCost)

$$c(d) = \begin{cases} 254 & d = 0 \\ 253 & 0 < d \leq r_{in} \\ 252e^{-\kappa(d-r_{in})'} & r_{in} \leq d \leq r_{inf} \\ 0 & d > r_{inf} \end{cases}$$

- d is the distance from the cell centre to the nearest obstacle cell.

- $r_{in} = 0.22m$ the inscribed radius (the robot's radius)
- $\kappa = 3.0$ the cost_scaling_factor
- $r_{inf} = 0.55m$ the inflation_radius

Global Planner

The global planner is NavFn, configured with use_astar: false, which makes it Dijkstra's algorithm on the costmap.

NavFn first converts each costmap cost c into a per-metre travel cost:

$$\tilde{c} = 50 + 0.8c$$

Dijkstra's recursion for the potential P

$$P(goal) = 0, \qquad P(c) = \underbrace{min}_{n \in neighbors\,(c)} [P(n) + l(c,n) \cdot \tilde{c}]$$

Where $l(c,n)$ is the distance between the centers of the two cells. Dijkstra's algorithm evaluates the cells in order of increasing P (with a priority queue), so each cell is finished exactly once. The path is then extracted by gradient descent on P from the start: move to the neighbour with the smallest potential until the goal is reached.

Local Planner

The global path is a list of points. The controller turns it into velocity commands 20 times per second. It uses DWB, Nav2's implementation of the dynamic window approach.

Every 50 ms, DWB tries 400 possible velocity commands. For each one it simulates 1.7 s of driving, scores the resulting trajectory and sends the best command.

Dynamic Window

$$v\,\epsilon[v_c - a_{dec}\Delta t, v_c + a_{acc}\Delta t\,] \cap [0, 0.26]$$

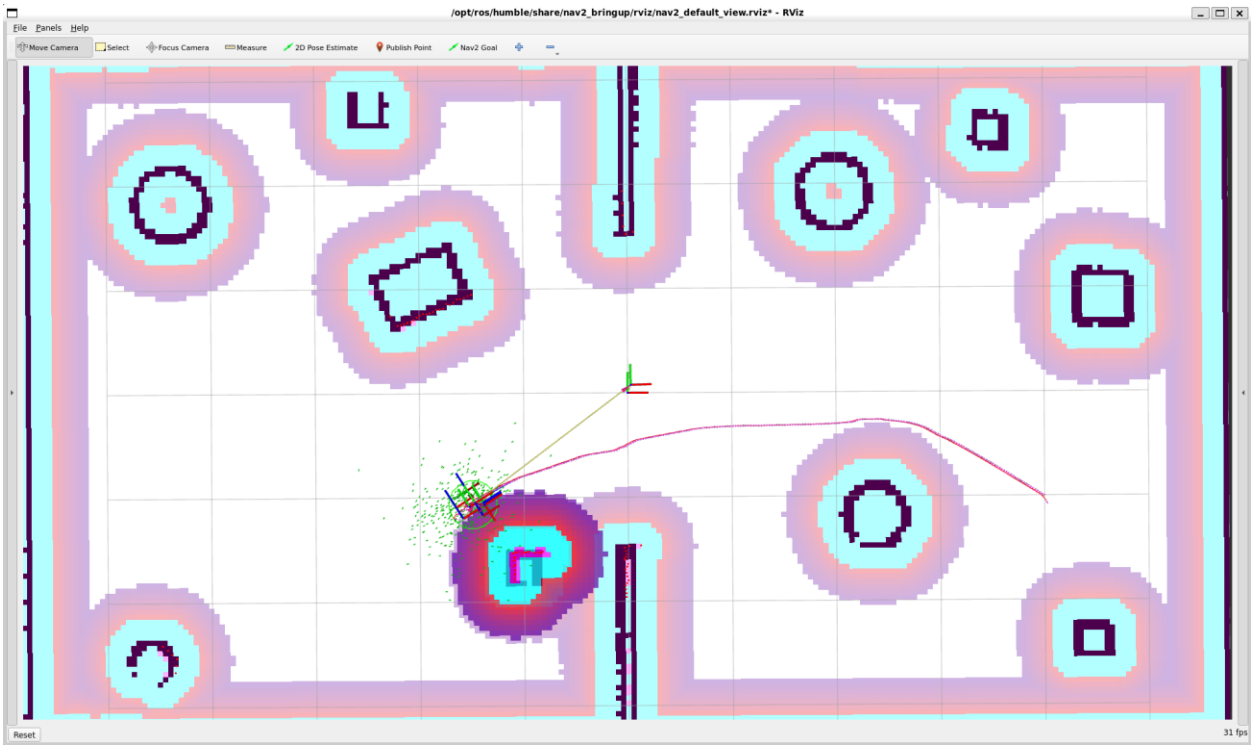
$$\omega\,\epsilon[\omega_c - \alpha_{dec}\Delta t, \omega_c + \alpha_{acc}\Delta t\,] \cap [-1.0, 1.0]$$

For a candidate (v, ω) held constant for T = 1.7s (sim_time). Points are generated every 5 cm of travel or 0.025 rad of turning (linear_granularity, angular_granularity).

The cost of these points are then scored and then the velocity and angular velocity is updated

$$J(v,\omega) = \sum_k s_k C_k(trajectory), \qquad (v*, \omega*) = arg\,min\,J(v,\omega)$$

- C_k is the cost given by critic k (path distance, goal distance, obstacle cost etc)
- s_k is the weight saying how much that opinion counts



All of this has to finish within one control period of 50ms.

Fig: The map of the simulation in Rviz showing the cost map and the path to a goal. It also shows the green particles the robot uses for the localization.

Automatic Dataset Labelling

For every picture, three things are known exactly: where the robot stood (the pose), where each object is (the world file) and how the camera maps the world to pixels. So, the project can predict where each object must appear. It then compares the prediction with the depth image the simulator produced, to make sure the object really is visible and not hidden behind something else. Finally, an independent color test confirms that the labelled pixels really have the color of the object. Only pictures that pass all checks go into the dataset.

The teleprot.py randomly teleports the robot to a chosen pose by calling the service /gazebo/set_entity_state, then the program waits 0.8 simulated seconds for the physics to settle and checks that the robot really is within 2 cm and 1.5° of the target pose before it saves anything which are then saved.

If poses were purely random, most pictures would show empty walls and floor, and the rare objects (especially the single red box) would be hardly seen. So, the program mixes three modes. The collected result is given below.

Mode	Weight	Frames collected (out of 1500)
red	35%	536
blue	35%	519
random	30%	445

In red and blue mode the program picks a free cell 0.6 to 5.0 m away from the chosen object with a clear line of sight and turns the robot toward the object with a random aiming error.

Camera Mechanics

Projection of a 3D Photo to a Pixel

$$u = f \cdot \frac{X}{Z} + c_x, \qquad v = f \cdot \frac{Y}{Z} + c_y$$

Projection in Matrix form

$$Z \cdot \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

Ray Casting

To predict where an object will appear, the project turns the problem around: it sends one ray through every pixel of the picture and asks which object the ray hits first. This is ray casting.

A camera loses one dimension: a pixel tells the direction of the point but not its distance. The depth image of the RGB-D sensor provides the missing distance Z for every pixel, so the 3D point can be rebuilt:

Ray of a Pixel

$$\boldsymbol{d_{cam}} = \begin{pmatrix} \dfrac{\boldsymbol{u - c_x}}{\boldsymbol{f}} \\ \dfrac{\boldsymbol{v - c_y}}{\boldsymbol{f}} \\ \boldsymbol{1} \end{pmatrix}, \boldsymbol{D = R_{wc} d_{cam}}$$

Where R_{wc} is the camera's orientation in the world derived from where the robot R_{wb} is currently at where the camera is mounted at the robot R_{bc}

$$\boldsymbol{R_{wc} = R_{wb} R_{bc}}, \qquad \boldsymbol{t_{wc} = R_{wb} t_{bc} + \begin{pmatrix} x \\ y \\ 0 \end{pmatrix}}$$

Ground Truth to Labelled Box

For every object in the world file, the program ray-casts through all 640*480 pixels and obtains the expected depth at every pixel that the object covers, and infinity elsewhere. Only depths between the near and far clip planes of the camera (0.1 m to 8.0 m) count, because nothing outside them is drawn by the simulator.

The object might be hidden behind a wall, a sphere or another object. The simulator's own depth image tells the truth: if something stands in front of the object, the measured depth at that pixel is smaller than the expected one. So, a pixel is counted as visible only if the two depths agree.

The object is then labelled into 3 categories

$$\boldsymbol{status} = \begin{cases} \boldsymbol{HIDDEN} & \boldsymbol{n_{vis} = 0} \\ \boldsymbol{LABEL} & \boldsymbol{n_{vis} \geq 80 \;\&\; f \geq 0.35} \\ \boldsymbol{AMBIGIOUS} & \boldsymbol{\text{visible but too small or too occluded}} \end{cases},$$

The detector's training (YOLO) stores one line per object: the class number and the box center, width and height, divided by the picture size so that all numbers lie between 0 and 1:

$$x_c = \frac{x_0 + x_1}{2W}, \qquad y_c = \frac{y_0 + y_1}{2H}, \qquad w = \frac{x_1 - x_0}{W}, \qquad h = \frac{y_1 - y_0}{H}$$

Data Splitting

The floor is divided into square cells of 1.5 m (according to where the robot stood), and whole cells are assigned to the training, validation or test set. A test picture is then always taken in a cell where no training picture was taken. This prevents near-duplicate pictures leaking between train and test.

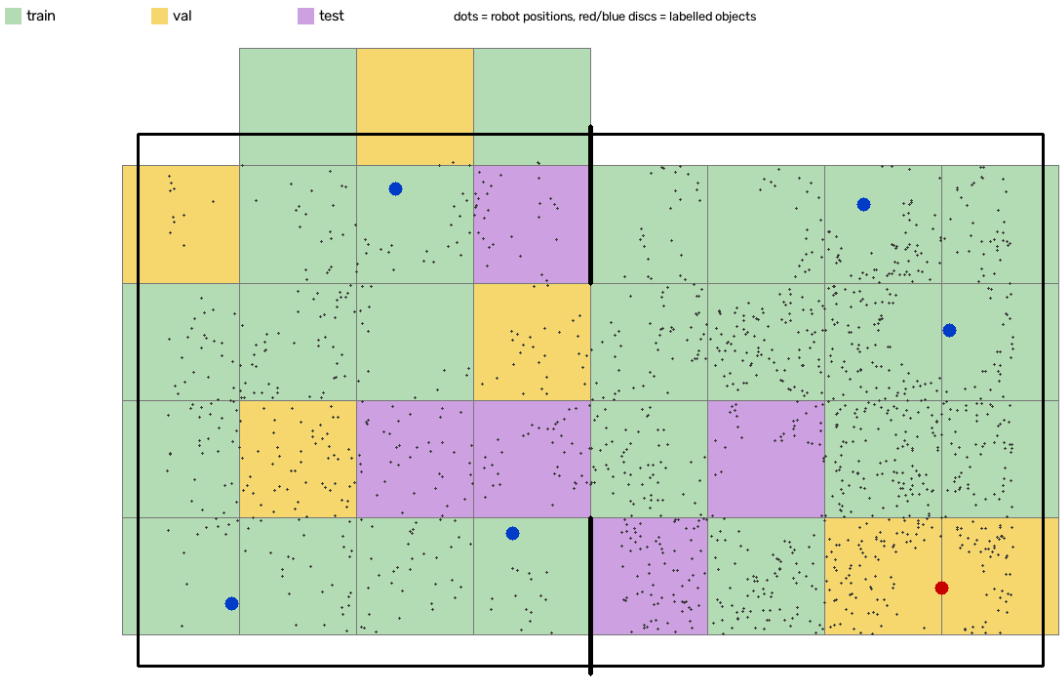


Fig: How the map was splitted

Convolution Networks and YOLO26

A convolution takes a small grid of numbers (filter or kernel) and slides it over the picture. At each position it multiplies the filter with the pixels beneath it and adds up the products. The result is large where the picture looks like the filter and small elsewhere. The network learns the filter numbers during training.

Convolution

$$y_o(i,j) = b_o + \sum_{c=1}^{C_{in}} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} W_{o,c,p,q}\ x_c(si + p - pad, sj + q - pad)$$

- x_c is the input channel c
- $W_{o,c,p,q}$ are the learned filter values for the output channel o
- **k** is the filter size
- **s** is the stride (how many pixels the filter jumps)
- **pad** is the border padding
- b_o is the offset

After each Convolution the batch is normalized

$$\hat{y} = \gamma \frac{y - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$$

- μ, σ is the mean & standard deviation computed from the training batch
- γ, β are learned variables used to rescale

Then the output passes through a SiLU Activation function.

$$SiLU(x) = x\sigma(x) = \frac{x}{1 + e^{-x}}$$

YOLO26 Network

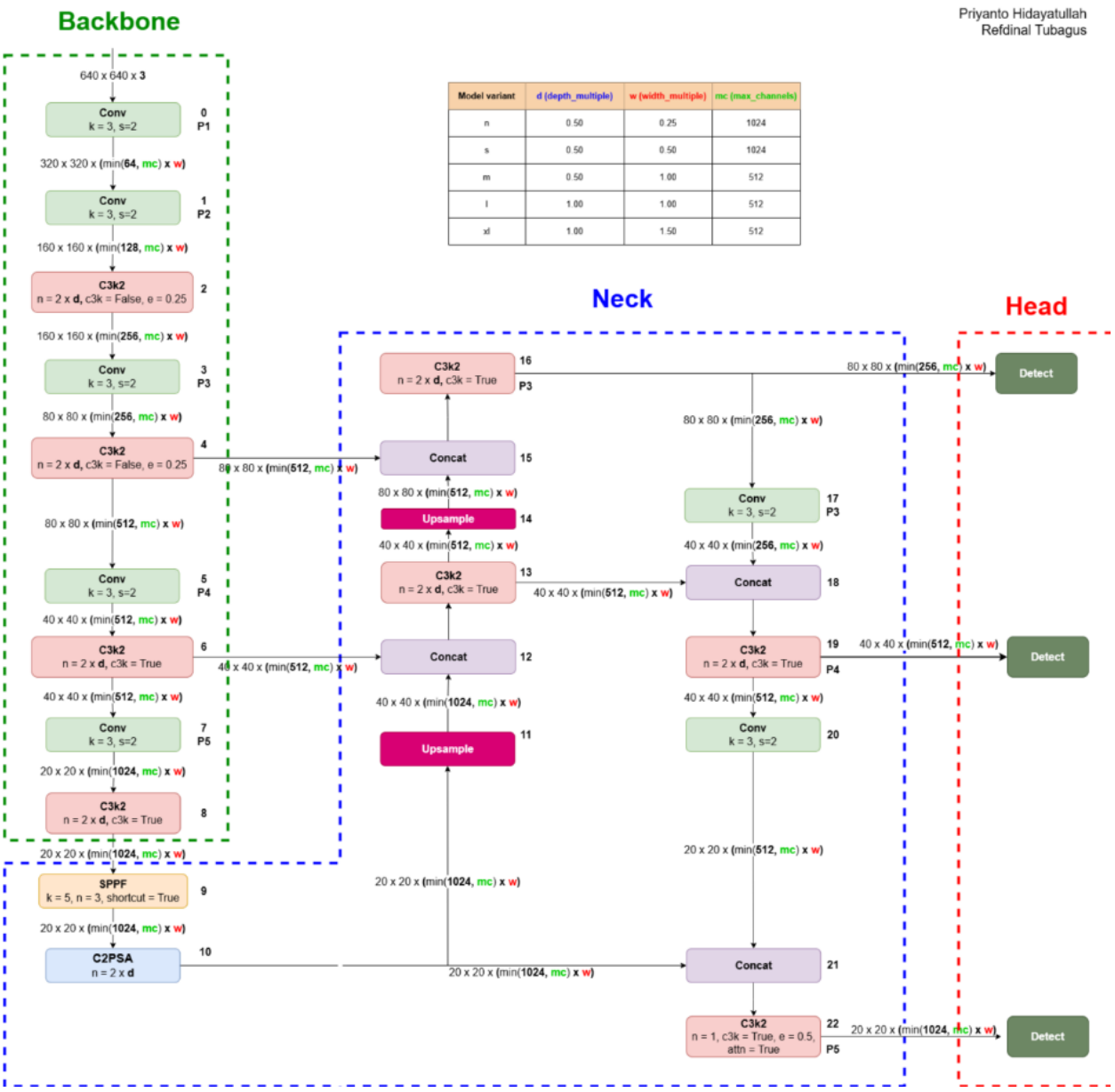


Fig: YOLO26 Architecture [1]

The YOLO (You Only Look Once) network is divided into 3 parts:

- Backbone: Extracts feature of increasing abstraction and decreasing resolution.
- Neck: Fuses features extracted by the Backbone.
- Head: Makes a Final Prediction

Training & Finetuning

At each location of each of the three feature maps, the head outputs $\mathbf{4} + \boldsymbol{n_c} = \mathbf{7}$ numbers (4 box numbers, 3 class scores). The number of locations for a 640*480 picture is

$$\underbrace{\mathbf{80} * \mathbf{60}}_{\boldsymbol{stride\ 8}} + \underbrace{\mathbf{40} * \mathbf{30}}_{\boldsymbol{stride\ 16}} + \underbrace{\mathbf{20} * \mathbf{15}}_{\boldsymbol{stride\ 32}} = \mathbf{6300}$$

Each location is a point at the center of its cell, and the four box numbers are the distances from that point to the four sides of the box, measured in units of the stride.

For every labelled object, the loss first decides which locations are responsible for it. It considers only locations whose center lies inside the object's box and ranks them by how well they already predict that object. For every location and class, the score is compared with its target and adds up the penalties.

Starting from the pretrained yolo26n.pt (696 of 708 weight tensors transferred), the network was trained for 40 epochs with AdamW and a linearly decaying learning rate.

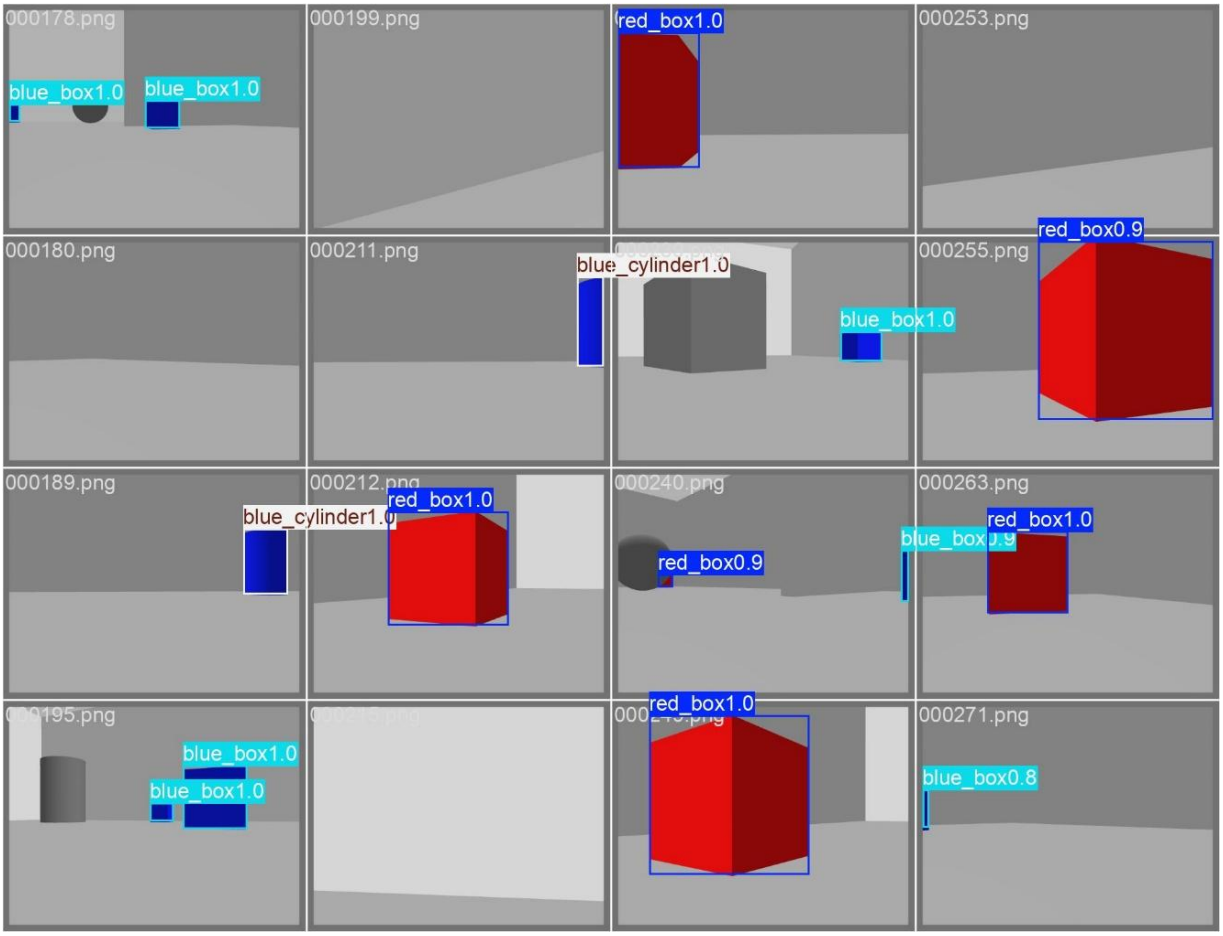


Fig: Prediction by the Finetuned YOLO26

Results Before & After

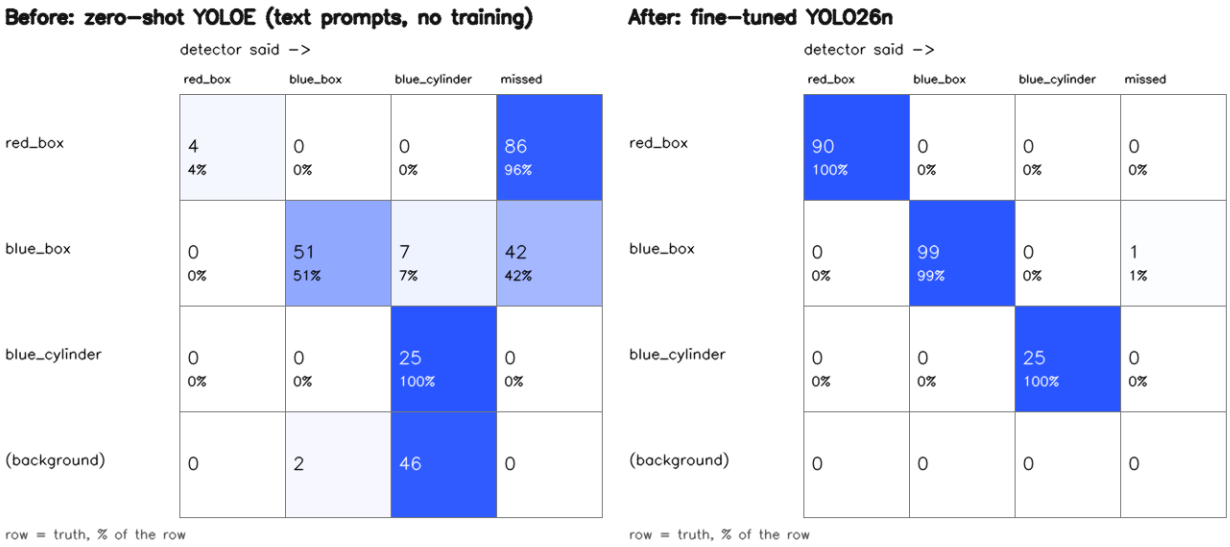


Fig: Confusion Matrix showing the results before and after finetuning the model

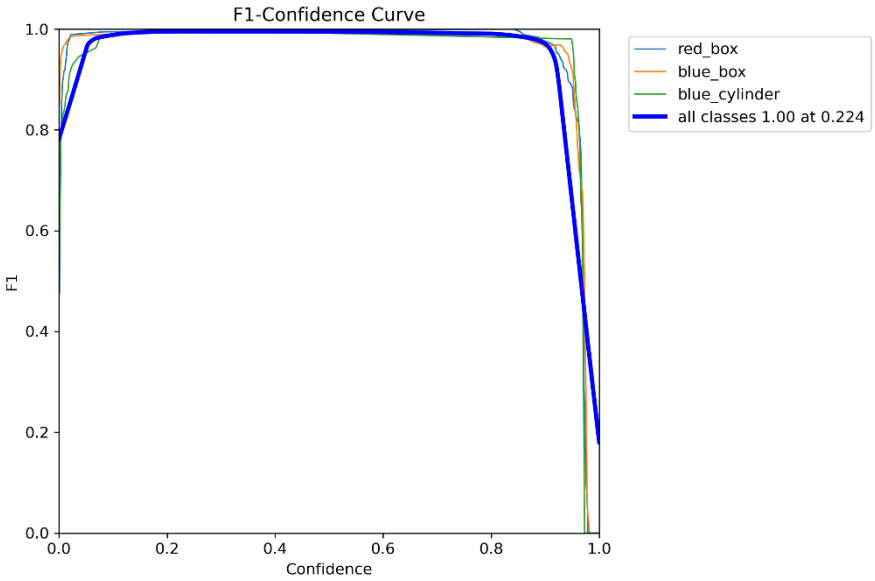


Fig: F1 Score after Finetuning

ROS2: Architecture, Nodes, Communication

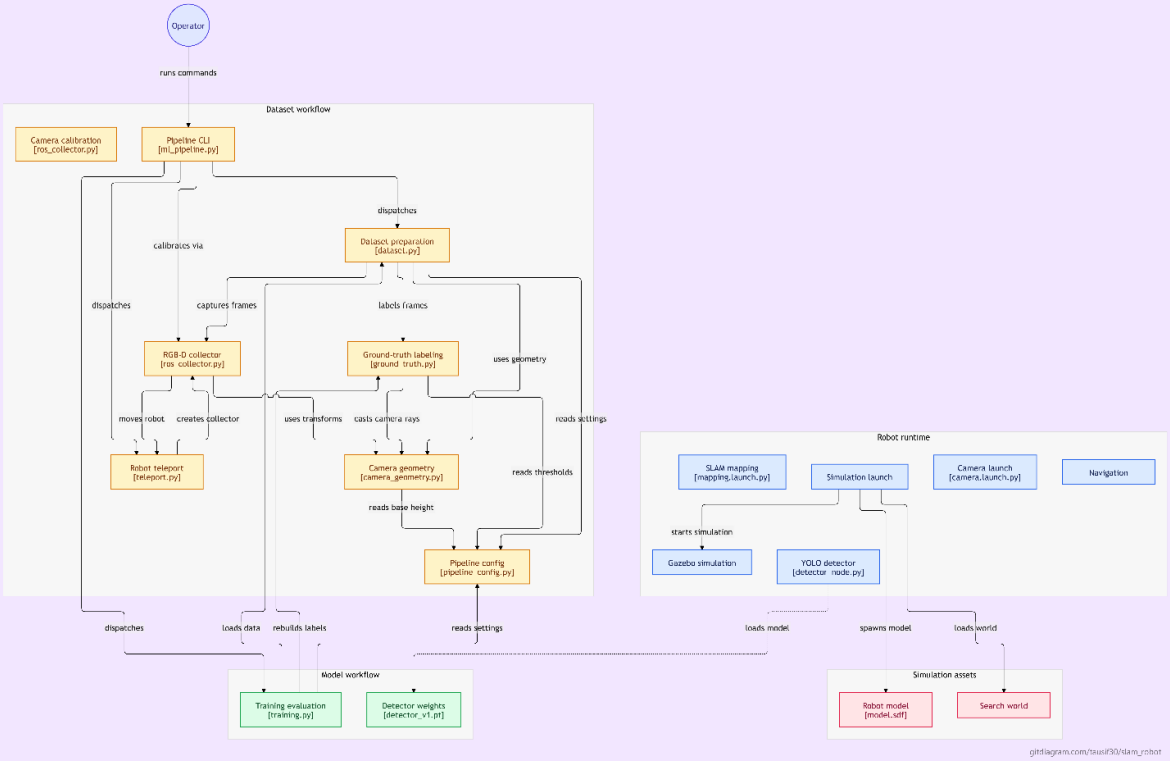


Fig: Runtime Architecture (From running my repo in gitdiagram)

Launch Files

Launch Files	What it does
simulation.launch.py	Launches the Gazebo World
navigation.launch.py	Starts Nav2 and RViz: map, costmaps, the Nav2 Goal tool
camera.launch.py	Shows the Robot's POV and the Annotations from the YOLO26
mapping.launch.py	SLAM Toolbox used to map the Gazebo World

Communication Types

ROS2 has 4 kinds of Communication Types:

Name	Communication Method	Purpose
Topic	Unidirectional & Asynchronous	Sending Data to Multiple Nodes
Service	Bidirectional & Synchronous	Request & Reply
Action	Bidirectional & Asynchronous	Task that takes long time
Parameter	Bidirectional & Synchronous	Dynamically changing behaviour

Tables of all the Communication Protocols of the Project is available in GitHub Repo.

Docker Image

The project needs ROS 2 Humble, Gazebo 11, Nav2, SLAM Toolbox, PyTorch and Ultralytics at matching versions, on a specific Ubuntu release. A fresh install of all of that takes hours and breaks easily. A Docker image is a frozen copy of a complete working system: whoever downloads it gets exactly the system that was tested.

The image was built in 4 stages

Stage	What it adds
Base	ROS 2 Humble & Packages: Gazebo ROS packages, TurtleBot3 meshes, Nav2, SLAM Toolbox, RViz 2, rqt_image_view, Python venv tools, OpenGL libraries, fonts
Python	Python Virtual Environment with NumPy 2.2.6, PyTorch 2.14.1 (CPU), Ultralytics 8.4.176
Workspace	ROS package, built with colcon
Runtime	Project Commands

A second Docker Compose file passes the Windows GPU into the container so that it draws Gazebo and Rviz. The neural network itself still runs on the CPU

Measurement	**CPU Drawing**	**GPU Drawing**
Gazebo RTF	0.85	1
Mean inference time per picture	1101 ms	99 ms
Pictures analysed per 10 s	8-10	53-62
RTF	0.85-0.90	1.0

Limitations

- As WORLD Value was used for the odometry_source, there was no drift, no wheel slip; loop closure has nothing to correct; AMCL starts from the exact pose. We need to use the real Odometry Method for real world application.
- The detector was tested on new positions, not on new appearances; the transfer to a real camera is untested. This also makes the task easier than real world object recognition.
- Map accuracy is limited by the 5 cm cells.
- The result was not reproduced in another machine (WSL Ubuntu 22.04).

Reference

[1] P. Hidayatullah and R. Tubagus, “YOLO26: A Comprehensive Architecture Overview and Key Improvements,” arXiv preprint arXiv:2602.14582, Feb. 16, 2026. https://doi.org/10.48550/arXiv.2602.14582